# EEEN30160 – Biomedical Signal Processing

# Assignment 2

Oisín Hogan

19355123

# Contents

Problem 1: 3
Code: 3
Output 3
Discussion 4
Problem 2: 5
Code: 5
Outputs: 6
Discussion: 6
a) 6
b) 7
Problem 3: 8
Code: 8
Output: 8
Discussion 9
Problem 4: 10
Code: 10
Output: 10
Discussion: 11
Problem 5 12
Code: 12
Output: 14
Discussion: 14

# Problem 1:

## Code:

```
fs = 48; % sampling frequency
t = 12; % time
intl = 6.73; % start of elevated part
intu = 9.52; % end of elevated part
aint = 3.5; % value of elevated part
tstep = 1/fs; % time step
T = 0:tstep:t; % time array

x = zeros(size(T)); % create signal as specifiec
intlt = find(abs(T - intl) < 0.01);
intut = find(abs(T - intu) < 0.01);
x(intlt:intut) = aint;

n = length(x);
y = abs(fft(x)); % let y = the filtered x
fy = (0:n-1)/n; % the frequencies of y
phase  = fftshift(y); % get the phase of y
fphase = [-n/2: n/2-1]/n; % get frequencies of phase of y

tiledlayout(3,1)
nexttile % plot x, y and phase
plot(x)
title('Original signal')
xlabel('time (s)')
nexttile
plot(fy, y)
title('Signal Magnitude')
ylim([0, 500])
xlabel('frequency (Hz)')
nexttile
plot(fphase, phase)
title('Signal Phase')
ylim([0, 500])
xlabel('frequency (Hz)')
```

## Output

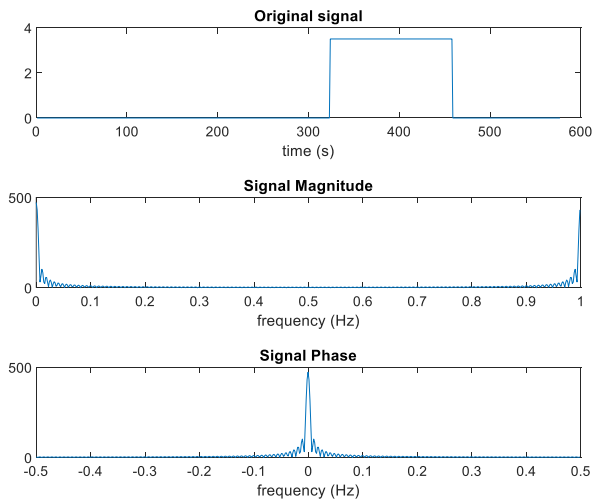


*Figure 1*

## Discussion

The magnitude of the ‘fft()’ of the signal is the same as the phase of the signal, but a phase apart. Both are symmetrical around the y axis.

# Problem 2:

## Code:

```
fcos = 2.38; % frequency of sine wave
t = 10;
tsamp = 1/100; % sampling time
T = [0:tsamp:t];
offset = 23; % phase offset in degrees
x1 = 4*cosd(fcos*T*180 + offset); % cosine array
n1 = 1000; % lower sampling frequency
n2 = 4000; % higher sampling frequency
x11k = abs(fft(x1, n1)); % filtered for magnitude of x1 at 1000 Hz
x14k = abs(fft(x1, n2));% filtered for magnitude of x1 at 4000 Hz
f1 = [0:n1-1]/n1; % Frequencies range of 1
f2 = [0:n2-1]/n2;% Frequencies range of 2
tiledlayout(3,2)
%plot x1 at 1000Hz sample
nexttile
plot(f1, x11k, 'b'),ylim([ 0 2100])
title('x1 sampled at 1000Hz')
nexttile
%plot x1 at 400Hz sample
plot(f2, x14k, 'r'),ylim([0 2100])
title('x1 sampled at 4000Hz')

fcos2 = 2.5;% 2nd cosine frequency
offet2 = 45; % second cosine phase offset
x2 = 3*cosd(fcos2*T*180 + offet2); % x2
xtot = x1+x2; % calculate xtot
xtot1k = abs(fft(xtot, n1)); % sampled at 1000Hz
xtot4k = abs(fft(xtot, n2));% sampled at 4000Hz

nexttile
%plot xtot at 1000Hz
plot(f1, xtot1k, 'b'),ylim([0 1600])
title('xtot sampled at 1000Hz')

nexttile
%plot xtot at 4000Hz
plot(f2, xtot4k, 'r'),ylim([0 1800])
title('xtot sampled at 4000Hz')
% calculate the phase of xtot for both sample frequencies
xtot1k = fftshift(xtot1k);
xtot4k = fftshift(xtot4k);
% plot phase diagrams of xtot
nexttile
plot(f1, xtot1k, 'b'),ylim([0 1700])
title('Phase of xtot sampled at 1000Hz')
nexttile
plot(f2, xtot4k, 'r'),ylim([0 1700])
title('Phase of xtot sampled at 4000Hz')
```

## Outputs:

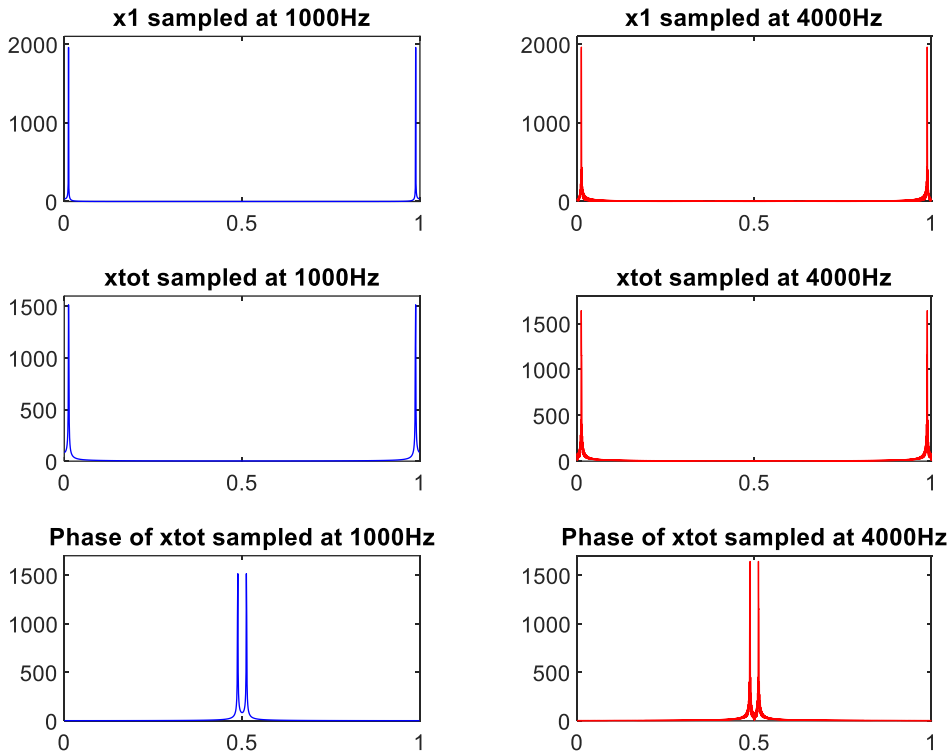


*Figure 2*

## Discussion:

a)

The higher N means that x1 gets a smoother increase in the phase magnitude, as can be seen clearly in the diagram below:

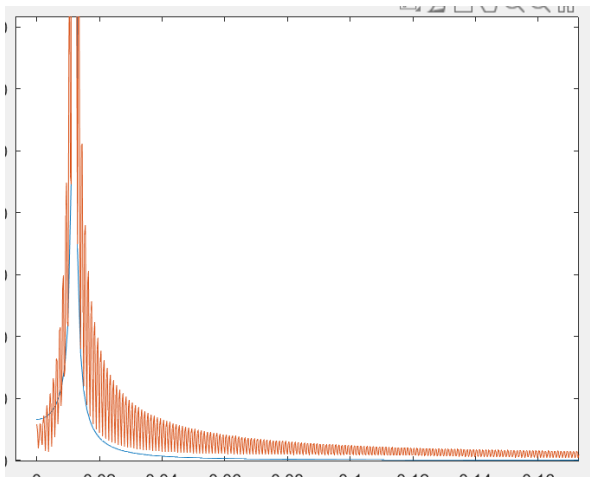

*Figure 3*

## b)

Similar to before, the higher N resulted in there being a smoother increase in the phase-magnitude diagram of the signal, where the changes were more exaggerated.

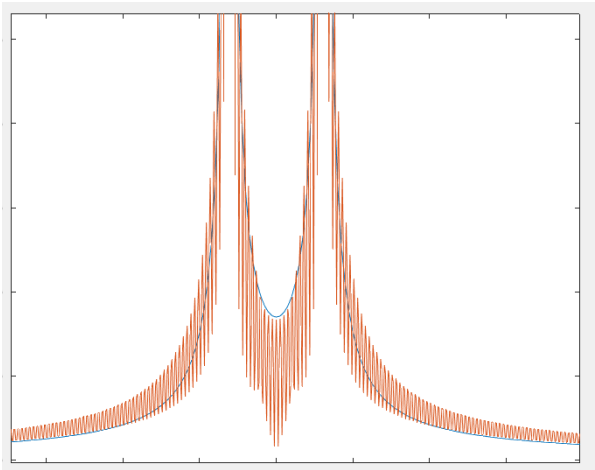

*Figure 4*

# Problem 3:

## Code:

```
load("C:\Users\Oisin\Documents\engineering_work\Signal\Lab 2\ecg.mat"); %
load file
fsamp = 500; %sampling frequeny
amp = 500; % amplication factor
ecg = (ecg/amp)*1000; %scale ecg
Necg = length(ecg); % get the length of the ecg
t = [0:1/fsamp:(Necg(1)-1)/fsamp]; % get a time array

tiledlayout(3,1)
nexttile % plot the ecg
plot(t, ecg)
title('ECG signal')
xlabel('Time (s)')
ylabel('Voltage (mv)')

nexttile % plot 1 period of the ecg, form 0.344s to 1.324s
plot(t, ecg), xlim([0.344 1.324])
title('ECG signal - 1 Period')
xlabel('Time (s)')
ylabel('Voltage (mv)')

ecgf = abs(fft(ecg, Necg)); %filter ecg
ecgf = fftshift(ecgf); % calculate the frequencies of the ecg
F=[-Necg/2:Necg/2-1]/Necg; %frequency of the phase
Famplified=F*500; % amplify frequency

nexttile % plot frequencies of wave form 0 - 60 Hz
plot(Famplified, ecgf)
title('ECG Frequencies')
xlim ([0 60])
```

## Output:

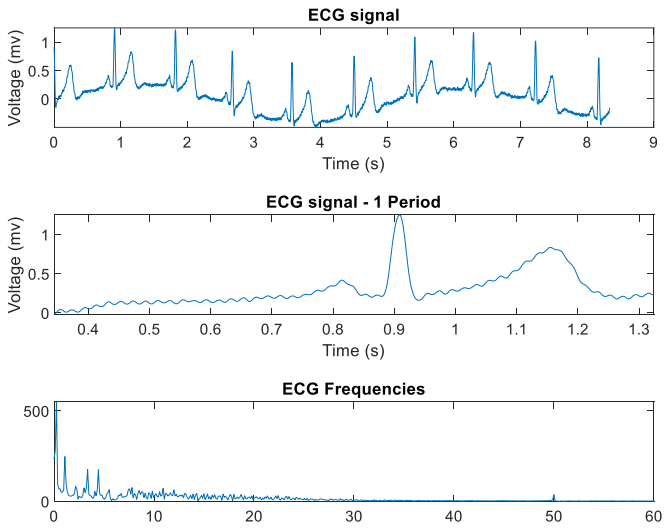


*Figure 5*

## Discussion

Looking at the third plot outputted by the code, it can be seen that there are several spikes of frequencies. The power line noise can be seen at 50 Hz.

The period time of the large movement is approximately 5.1 seconds, giving it a frequency of about 0.2Hz.

The period of the heartbeat pulse is about 1Hz, calculated from the observed period being 0.98s.

# Problem 4:

## Code:

```
len = 20; % length of vector
x = zeros(len,1); % initalise impulse
x(1) = 1; % add impulse
y = zeros(len,1); % initialise ouput vector
for n = (1: len) % loop to fill out signal response
    if n == 1 % used for the first n-1 term
        y(n) = 0.98*x(n);
    else % used for every n-1 term after the inital one
        y(n) = .98*x(n) + 1.83*x(n-1) - .92*y(n-1);
    end
end
% plot graph
plot(x, 'r-')
hold on
plot(y, 'b-')
```

## Output:

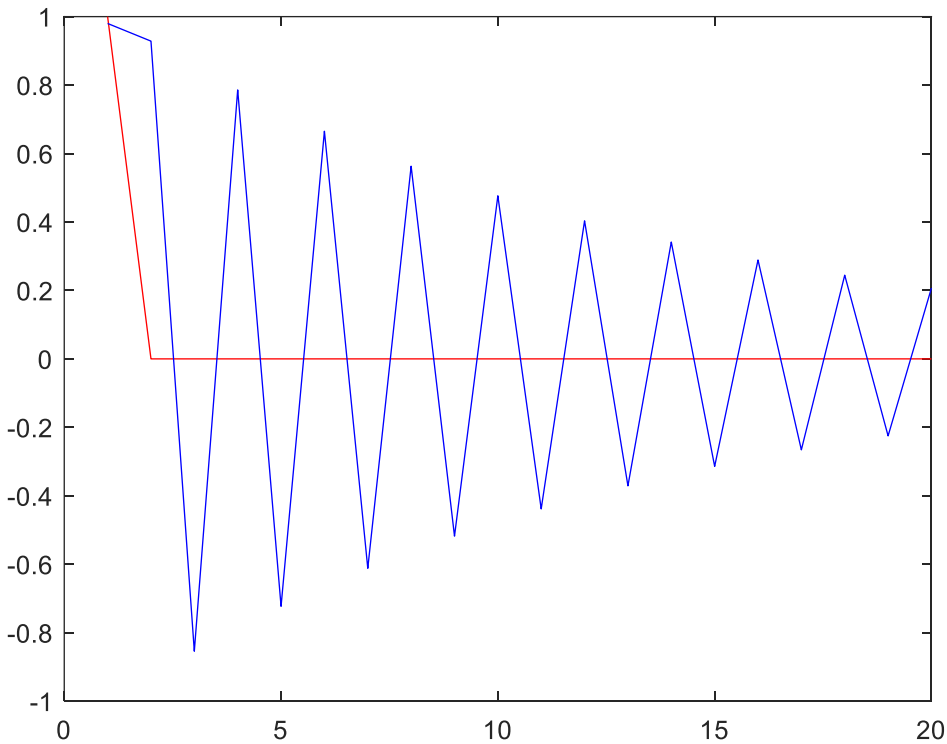


*Figure 6*

## Discussion:

This is an IRR filter as it utilised current and past inputs as well as past outputs in order to filter the impulse.

B1 = 0.98 B2 = 1.83 A1 = 0.92

## Problem 5

### Code:

```
%for impulse
len = 10; % array length
x1 = zeros(len,1); % initalise x1
x1(1) = 1; % add impulse
y1 = zeros(len,1); % initialise output 1

for n = (1: len) % for loop to calculate output
    if n == 1 % if statements used to account for x(-3), x(-2), and x(-1),
letting them equal 0
        y1(n) = x1(n);
    elseif n == 2
        y1(n) = x1(n) + 1.2*x1(n-1);
    elseif n == 3
        y1(n) = x1(n)+ 1.2*x1(n-1) + .1*x1(n-2);
    else
        y1(n) = x1(n) + 1.2*x1(n-1) + .1*x1(n-2)+ 0.01*x1(n-3);
    end
end
% for cosines
fs = [0.5, 20, 100]; % arrays of frequencies
fsamp = 250; % sampling frequency
tstart = 0; % starting time
tend = 3; % finishing time
t = (tstart:1/fsamp :tend); % time array
sig1 = cos(2*pi*t*fs(1)); % signal 1
sig2 = cos(2*pi*t*fs(2)); % signal 2
sig3 = cos(2*pi*t*fs(3)); % signal 3

x2 = [sig1; sig2; sig3]; % array of signals
y2 = zeros(size(x2)); % array of outputs
len = length(sig1); % get length of signal and set to length
for m = 1:3 %for loop used to do all three signals efficiently
    for n = (1: len) % same process to calculate outputs as was done above
        if n == 1
            y2(m,n) = x2(m,n);
        elseif n == 2
            y2(m,n) = x2(m,n) + 1.2*x2(m,n-1);
        elseif n == 3
            y2(m,n) = x2(m,n)+ 1.2*x2(m,n-1) + .1*x2(m,n-2);
        else
            y2(m,n) = x2(m,n) + 1.2*x2(m,n-1) + .1*x2(m,n-2)+ 0.01*x2(m,n-
3);
        end
    end
end
% plot impulse response
figure(1)
plot(x1, 'r-')
hold on
plot(y1, 'b-')
title('Filtered response to Impulse')
%plot cosine response, in a separate figure
figure(2)
tiledlayout(3,1)
nexttile
plot(x2(1,:), 'r-')
hold on
plot(y2(1,:), 'b-')
```

```
title('Filtered response to cosine wave of f = 0.5Hz')

nexttile
plot(x2(2,:), 'r-')
hold on
plot(y2(2,:), 'b-')
title('Filtered response to cosine wave of f = 20Hz')

nexttile
plot(x2(3,:), 'r-')
title('Filtered response to cosine wave of f = 100Hz')
hold on
plot(y2(3,:), 'b-')
```

## Output:

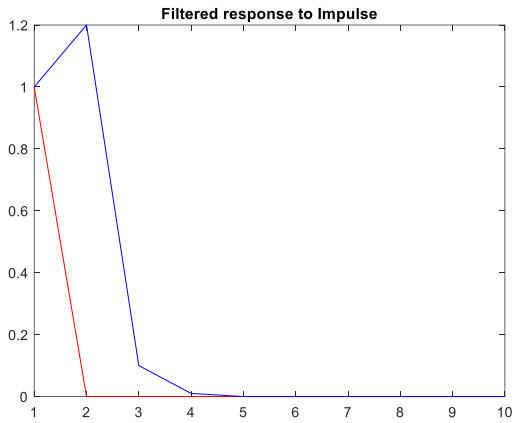


*Figure 7*

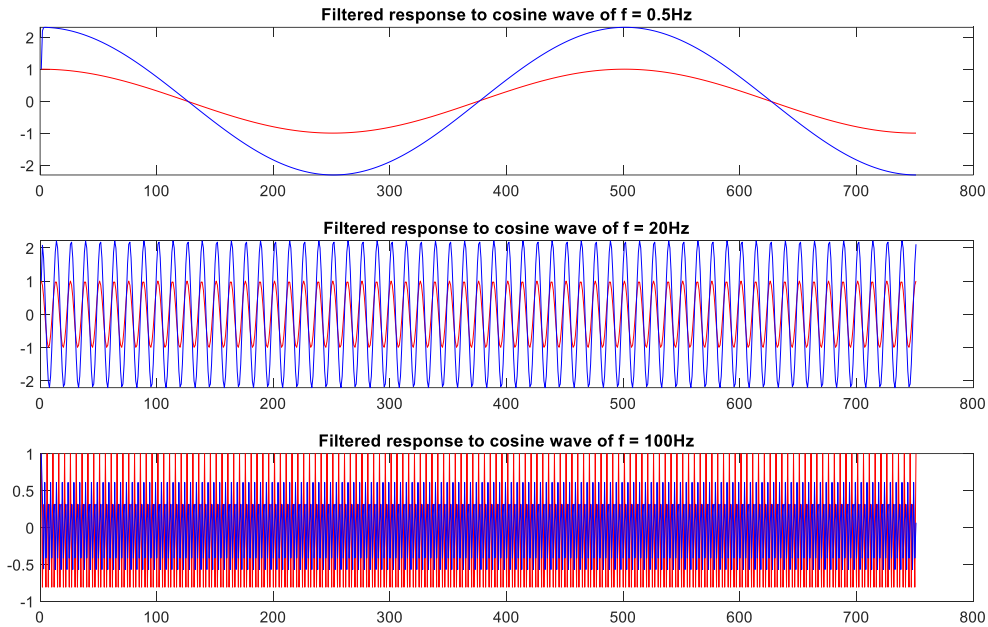


*Figure 9*

## Discussion:

This filter is an FIR filter as it requires only the current and previous inputs in order to calculate the outputs, no previous outputs are needed.

As can be seen above, where the input is in red, and the output in blue, in the first 2 plots the filter can be seen amplifying the cosine wave, but in the third, it seems to have reduced the amplitude.